DAFN26

PROGETTI FINALI

Ultimo aggiornamento: 29 Settembre 2026 (Precisazione requisiti generali)

INDICAZIONI

Ciascun progetto deve essere svolto da un gruppo di persone di numero pari a quello indicato.

Per ciascuna traccia, si può fare riferimento ai seguenti supervisori.

- Roberto Masocco
- Alexandru Cretu
- Alessandro Tenaglia

Gli studenti sono liberi di sviluppare le soluzioni che ritengono migliori per ogni traccia, anche ignorando i suggerimenti forniti o aggiungendo ulteriori funzionalità, **purché siano rispettati tutti i requisiti ufficialmente elencati**.

Possiamo fornirvi ulteriori esempi di codice, spiegazioni e documentazione qualora vi siano utili.

Il progetto si considera completato quando il componente software richiesto presenta tutte le funzionalità previste e soddisfa tutti i requisiti.

L'uso di AI per code generation deve essere **consapevole e responsabile**. È consigliato l'uso di AI Claude/ChatGPT per **raccogliere spiegazioni e suggerimenti** relativi ad API specifiche da usare, ad es. a corredo della documentazione ufficiale. **È sconsigliato l'uso di strumenti di "vibe coding" o altra forma di coding automatizzato o AI-assisted quali Claude Code, Codex, etc.**

Quando è richiesta un'installazione di Linux, una VM **non** è sufficiente.

Come risorse generali a disposizione, si considerino

- tutto il materiale didattico fornito a lezione;
- la documentazione ufficiale di ROS 2 Jazzy, ricca di esempi e tutorial **sia in C++ che in Python**;

- l'esempio topic_pubsub_py di coppia publisher/subscriber in Python presente nel materiale didattico.

REQUISITI GENERALI

- Ogni progetto richiede l'implementazione di un **pacchetto software ROS 2**, **in C++ o in Python**.
- Tutti i progetti dovranno essere svolti sotto version control, entro repository Git hostate su **GitHub** a cui vi verrà fornito l'accesso dopo un setup iniziale (comprendente, e.g., le interfacce ad-hoc ed eventuali dipendenze aggiuntive necessarie). Con ogni gruppo verrà organizzato, appena possibile, un incontro iniziale. **Comunicare mediante foglio Excel che sarà condiviso**
  - username ed email GitHub dei componenti di ogni gruppo;
  - il progetto scelto da ciascun gruppo.
- Ogni package ROS 2 dovrà contenere, organizzati opportunamente secondo la struttura mostrata a lezione (si veda l'esempio di codice sui parametri mostrato nella relativa lezione):
  - codice sorgente;
  - config file in opportuna directory;
  - launch file in opportuna directory;
  - gestione parametri tramite librerie
    - params_manager_cpp per i package in C++;
    - params_manager_py per i package in Python;
  - compilazione nodo sia come eseguibile standalone che come component (solo per package C++).
- Tutti i progetti dovranno essere svolti in ambiente di lavoro gestito tramite **Docker**, usando il framework **Distributed Unified Architecture (DUA)** in uso nel laboratorio. Gli ambienti saranno forniti inizialmente pronti per essere usati. Per approfondirli, sono disponibili le seguenti risorse:
  - repository GitHub dua-template (su cui saranno basate quelle dei progetti);
  - quick-start guide dua-template;
  - tesi di Dottorato di R. Masocco capp. 4, 5.1 (cap. 3 costituisce un "libro di testo" per gli argomenti trattati durante le lezioni).

TRACCIA DEL PROGETTO

DRIVER SENSORE QUALITÀ ARIA (2 persone) (C++) (Linux richiesto)

Nodo driver per sensore di qualità dell'aria, interfacciabile via USB UART (seriale, basato su Arduino Uno R3).

- Funzionalità richieste
  - **Requisiti generali.**
  - Il nodo dovrà gestire
    - interfacciamento del sensore tramite libreria serial (fornita già installata nell'ambiente di lavoro);
    - acquisizione periodica dei dati, prodotti a 0.1 Hz in formato JSON con codifica UTF-8, **mediante thread dedicato**;
    - pubblicazione dei dati su topic di nuovo in formato JSON;
    - si consiglia l'uso della libreria nlohmann/json per la gestione dei dati in tale formato (fornita preinstallata nell'ambiente di lavoro).
  - Procedura di "abilitazione" del nodo:
    - eseguita quando viene richiesta l'abilitazione da servizio, o alla fine del costruttore del nodo se il parametro "autostart" è impostato a "true";
    - crea oggetti di libreria serial per aprire e leggere dalla porta seriale;
    - crea thread con routine dedicata per acquisire i dati, post-processarli, e pubblicarli su topic.
  - Procedura di "disabilitazione" del nodo:
    - eseguita quando viene richiesta la disabilitazione da servizio, oppure nel distruttore del nodo;
    - fa terminare e joina thread;
    - chiude porta seriale, rilasciando le relative risorse precedentemente acquisite.
  - Thread routine:
    - acquisisce un nuovo campione dalla porta seriale, in formato JSON;
    - esegue post-processing (si vedano risorse a seguire);
    - pubblica campione raw e post-processato in formato JSON (si vedano risorse a seguire).
  - Publishers
    - ~/readings (std_msgs/String)

        - Dump su stringa (campo “data”) del contenuto dell’ultima lettura dopo post-processing, di nuovo in formato JSON (si vedano le risorse messe a disposizione a seguire per informazioni sul formato e sugli step di post-processing).
    - ~/readings_raw (std_msgs/String)
        - Dump su stringa (campo “data”) del contenuto dell’ultima lettura così come è stata acquisita dal sensore.
- Servizi
    - ~/enable (std_srvs/SetBool)
        - A seconda del valore del campo “data”, abilita o disabilita il campionamento da parte del nodo come sopra specificato.
- Parametri
    - autostart (BOOL)
        - Avvia il campionamento già dall’interno del costruttore (suggerimento: controllarlo solo alla fine del costruttore, dopo che ogni altra entità è stata creata).
    - sensor.calibration.conv_CO (DOUBLE)
        - Default value: 0.4
        - Conversione da mV in ppb
    - sensor.calibration.m_VOC (DOUBLE)
        - Default value: 15.76
        - Conversione da Volt in ppm
    - sensor.calibration.q_VOC (DOUBLE)
        - Default value: 0.73
        - Conversione da Volt in ppm
    - sensor.calibration.RF_VOC (DOUBLE)
        - Default value: 0.5
        - Fattore di risposta PID
    - sensor.calibration.RO_CH4 (DOUBLE)
        - Default value: 1.5
    - sensor.device.baudrate (INTEGER)
        - Baud rate a cui configurare la porta seriale del sensore.
    - sensor.device.file (STRING)

        - Percorso completo del device file corrispondente alla porta seriale a cui è collegato il sensore (e.g. “/dev/ttyACM0”).
- Risorse disponibili
    - Indicazioni sui formati JSON e sul post-processing.

I test verranno eseguiti dagli studenti sul simulatore di sensore basato su Arduino Mega 2560 che gli verrà consegnato; gli studenti possono portarlo con sé e svolgere sviluppo e testing in autonomia.